

1 墙内本机 + 墙外 VPS: 自建 VLESS + REALITY 与 Hysteria2 节点

你的情况: 本机在中国大陆, 被 GFW 挡住, 访问不了墙外网站 (GitHub、Google、下载站都打不开); 手上有一台**境外的 VPS, 它有正常外网**。

这种情况是**最简单的一种**, 因为所有下载都能在服务器上完成。全文按这个场景组织, 主线只有四步:

步骤	做什么	章节
1	确认本机能 SSH 上 VPS, 并开一条 SOCKS5 通道 应急上网	§0.1 / §2
2	在服务器上装 Xray (VLESS+REALITY) + Hysteria 2, 下载全在服务器完成	§3 §4 §5
3	解决「客户端软件下不到」这个真实阻塞点	§7
4	导入节点、验收、排错	§6 §9 §10

资料核实时间: 2026-09。文中所有安装路径、配置字段均以官方仓库/官方文档为准 (Xray-core v26.3.27、sing-box 1.11+、Hysteria 2 官方安装脚本)。

1.1 0. 先搞清楚: 你的「没外网」是哪一种

GFW 环境下有三件容易混淆的事, 对应完全不同的解法。**先对号入座, 再往下读。**

编号	你的实际情况	是不是你的情况	真正的解法	章节
A	本机在墙内访问不了墙外网站, 但 VPS 在墙外、有正常外网	✓ ← 就是你的情况	服务器上 curl 一切正常。你完全不需要离线安装, 只需要一条能用的通道	§0.1
B	服务器本身没有外网 (内网机器、只给内网 IP、无出网权限)	✗ 不是 (除非你的 VPS 是这种)	在别的能上网的机器上预先下载二进制再传进去	§5.5
C	通道通了, 但客户端软件本身下不到 (本机装不了 v2rayN 等)	✓ 你多半也属于这类	用 SSH 自带的 SOCKS5 顶一阵, 再把客户端从服务器传回来	§7

1.1.1 核心思路 (记住这一条就够)

把「下载」和「连接」彻底分开。

下载 (GitHub、apt、ACME 证书) 全部发生在**服务器**上——它在墙外, 看 GitHub 毫无障碍。你要操心的只有**连接**: 从墙内本机到墙外服务器, 怎么建立第一条通道。

而这件事 SSH 自己就能做: -D 参数开一个 SOCKS5, **零依赖、零安装**, 你的浏览器立刻就能上墙外。后面所有配置工作都在服务器上敲, 你本机甚至不需要装任何代理软件就能完成部署。

1.1.2 0.1 第 0 步: 确认连接并开一条应急通道

你本机和 VPS 能正常建立连接 —— 那整件事的门槛就已经过了。这一步做两件事: **确认**, 然后把这条连接直接变成代理。

(1) 确认 SSH 通, 且服务器能出墙外

```
ssh root@<服务器IP> 'echo "== SSH 连通 =="; echo -n "服务器出口 IP: "; curl -s --max-time 8 https://api.ipify.org; echo'
```

代码已另存为 [snippets/01-0.1-第-0-步: 确认连接并开一条应急通道.sh](#) (1 行, bash)

看到「SSH 连通」加上一串 IP (就是你 VPS 的 IP) 就成了。

(2) 把这条 SSH 连接直接变成 SOCKS5 代理

```
ssh -D 1080 -N -f root@<服务器 IP>
```

代码已另存为 [snippets/02-0.1-第-0-步: 确认连接并开一条应急通道.sh](#) (1 行, bash)

-D 1080 在本机开一个 SOCKS5 端口，-N 不执行远程命令，-f 放到后台。不需要在服务器上装任何东西，也不需要在本机装任何客户端。

(3) 验证代理真的生效

```
curl -x socks5h://127.0.0.1:1080 https://api.ipify.org
```

代码已另存为 [snippets/03-0.1-第0-步：确认连接并开一条应急通道.sh](#) (1 行, bash)

返回的 IP 应该和 (1) 里服务器上 curl 出来的一致。浏览器怎么用它见 §2.2。

从这一刻起，你已经有一条能上墙外的通道了，虽然慢，但可用。

这意味着：后面所有部署工作都可以在服务器上完成，你本机不需要装任何代理软件。本节的 SOCKS5 通道请**全程保持**——它是你唯一的「逃生通道」：万一新节点配错了、把自己关在门外，靠它还能登回服务器修。

▲ 千万别在部署过程中把 SSH 改坏（改端口、改配置、换密钥）。SSH 是你唯一的救命通道，出问题就只能靠服务商控制台的 VNC 了。稳妥做法见 §3.2。

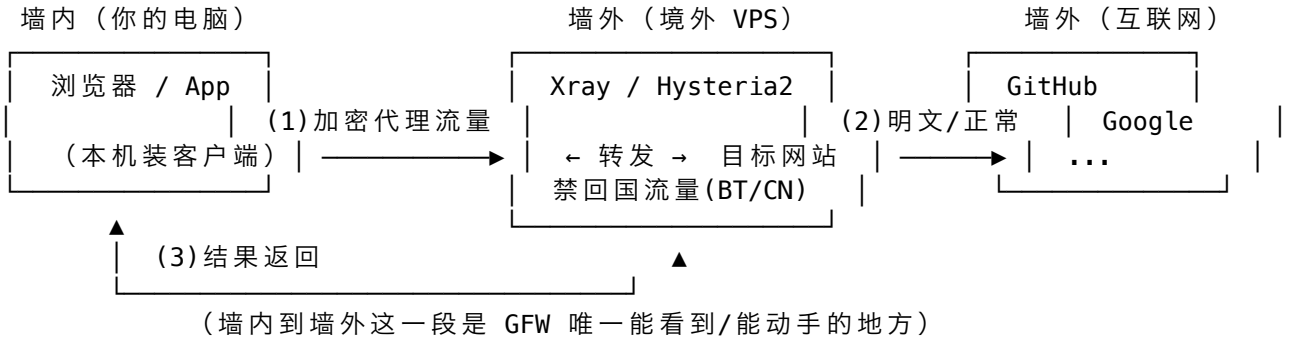
1.1.3 关于「一条通道都没有」

服务商网页控制台的 VNC / 网页终端只能用来敲命令改配置，**不能**把流量转发到你本地。别指望它当代理用。

1.2 1. 转发链路与协议选型

1.2.1 你要搭的东西，本质是一个「转发节点」

你的 VPS 扮演的角色是**流量中转**：本机的流量先到 VPS，再由 VPS 转发到墙外目标网站，响应原路返回。



代码已另存为 [snippets/04-你要搭的东西，本质是一个「转发节点」.txt](#) (10 行, text)

三个关键点，直接决定了后面的选型：

- 1. (1) 这一段必须扛住 GFW 的深度包检测 (DPI) 和主动探测。这是选 REALITY 的唯一理由。
- 2. (2) VPS 在墙外，上网完全自由——但如果你不回程也走墙外 (geoip:cn 直连)，会在链路上留下「境内 IP 主动访问境外代理服务器」这种明显特征。所以配置里要**禁回国流量**。
- 3. 本机必须装客户端，除非你只用 SSH SOCKS5 应急。SSH 隧道本质上也属于「(1) 这一段」，只是它走 22 端口、明文协议，**不适合长期使用**（慢、特征明显、不能给手机用）——它只是让你撑到正式节点上线。

1.2.2 选型：GFW 环境下为什么要 TCP 优先

	VLESS + REALITY	Hysteria 2 (hy2)
传输层	TCP（套在 TLS 1.3 里）	UDP / QUIC
需要域名	不需要	需要（TLS 证书；或自签绕过）
需要 DNS 解析	不需要（直接连 IP）	需要（域名要解析到该 IP）
抗主动探测	强。鉴权失败的握手会被转发到真实大站，探测者看到的是真站点	中。靠 Salamander 混淆伪装，UDP 流量本身特征明显
抗 DNS 污染	天然免疫（不查 DNS）	受影响（域名解析可能被污染）
晚高峰表现	稳	容易被 UDP QoS 限速/丢包
速度	中高，受 TCP 拥塞控制限制	网络好时很快（Brutal 主动发包）
主要风险	IP 被封（配置正确时概率低）	UDP 被限速/封锁；域名可能暴露

墙内网络对 UDP 最狠。GFW 和各运营商对 QUIC/UDP 的限速、丢包、封禁普遍比 TCP 严格得多，尤其是晚高峰和跨境链路。所以：

在墙内，TCP 线路是刚需，UDP 线路是加速。

结论：两个都装，VLESS + REALITY 当主力。

- VLESS + REALITY (TCP/443) = 主力 + 唯一兜底。任何网络环境都能跑，是你的保命线路。
- Hysteria 2 (UDP/443) = 加速。网络好的时候速度明显更快；被限速了也不影响你有网用。
- 客户端配成「故障转移 / 自动测速」组，UDP 不通时自动回落到 TCP 节点。
 - ▲ 别只装 hy2。墙内 UDP 一被 QoS，你连「能连上」都做不到。必须保留 TCP 线路，否则 hy2 一挂就彻底没网（只剩 SSH 应急通道能凑合）。

1.2.3 端口规划

端口	协议	用途
22（或自定义高位端口）	TCP	SSH 管理，只对你的 IP 开放最好
443	TCP	VLESS + REALITY
443	UDP	Hysteria 2
80	TCP	仅 ACME HTTP-01 验证需要（用 DNS-01 则可不开）

TCP 443 和 UDP 443 是两个完全独立的端口空间，可以同时被两个服务占用，互不冲突——这是本方案能省下大量端口的关键。

1.3 2. 应急通道：先让你自己能上网

这是本文最重要的一节。在装任何代理之前，先用 SSH 打通一条 SOCKS5 通道。

1.3.1 2.1 一条命令搞定

```
ssh -D 1080 -N -f root@<服务器 IP>
```

代码已另存为 [snippets/05-2.1-一条命令搞定.sh](#)（1 行，bash）

参数含义：-D 1080 在本地开一个 SOCKS5 代理端口；-N 不执行远程命令；-f 后台运行。

验证：

```
curl -x socks5h://127.0.0.1:1080 https://api.ipify.org
```

代码已另存为 [snippets/06-2.1-一条命令搞定.sh](#)（1 行，bash）

返回的应该是你服务器的 IP，不是空、不是报错。返回空说明通道没通。

socks5h:// 里的 h 在墙内是生死攸关的差别，千万别写成 socks5://。

GFW 对「发往墙外的裸 DNS 查询（UDP/53）」污染得很彻底——你 query 一个墙外域名，本地 DNS 服务器返回的可能是完全无关的 IP。

写法	谁做 DNS 解析	结果
socks5h://	代理端（你的 VPS）	✓ 域名由 VPS 解析，拿到真实 IP，正确
socks5://	本机	✗ 本机 DNS 被污染，拿到假 IP，连接失败或被劫持

h = hostname resolution happens on the proxy side。后面 §2.3 的 ALL_PROXY、apt、git、pip 全部沿用这个写法。

1.3.2 2.2 让浏览器用上它

Firefox（推荐，最省事，且自带 DNS 代理开关）

1. 地址栏访问 about:preferences
2. 搜索 network proxy → 「网络代理」→ 选「手动配置」
3. SOCKS 主机 127.0.0.1，端口 1080，SOCKS v5 勾上
4. ✓ 勾选「使用 SOCKS 代理时也使用 DNS 代理 DNS」（关键，否则 DNS 走本地会被污染）

Chrome / Edge

Chrome 原生不带代理设置开关，用扩展 SwitchyOmega（墙内可能下不到，见 §7），新建「代理服务器」协议选 SOCKS5、填 127.0.0.1:1080。

应急阶段用浏览器验证就够用了：能打开 GitHub / Google，说明通道没问题，可以放心开始装正式节点。

这一步不用非得先把客户端装好。

1.3.3 2.3 让命令行工具走代理

```
# 一次性
export ALL_PROXY=socks5h://127.0.0.1:1080

# apt
cat > /etc/apt/apt.conf.d/99proxy <<'EOF'
Acquire::http::Proxy "socks5h://127.0.0.1:1080";
Acquire::https::Proxy "socks5h://127.0.0.1:1080";
EOF

# git
git config --global http.proxy socks5h://127.0.0.1:1080

# pip
pip config set global.proxy socks5h://127.0.0.1:1080
```

代码已另存为 [snippets/07-2.3-让命令行工具走代理.sh](#) (14 行, bash)

1.3.4 2.4 保持连接不掉线

~/.ssh/config 里加一段, 以后 ssh vps 一条命令搞定:

```
Host vps
  HostName <服务器IP>
  User root
  Port 22
  ServerAliveInterval 30
  ServerAliveCountMax 6
  TCPKeepAlive yes
  Compression yes
  ExitOnForwardFailure yes
```

代码已另存为 [snippets/08-2.4-保持连接不掉线.txt](#) (9 行, text)

```
ssh -fN -D 1080 vps      # 开 SOCKS5
autossh -M 0 -fN -D 1080 vps  # 有 autossh 的话会自动重连
```

代码已另存为 [snippets/09-2.4-保持连接不掉线.sh](#) (2 行, bash)

1.3.5 2.5 把文件从服务器传回来 (拿不到软件时用这个)

```
# 方式一: scp, 最简单
scp root@<服务器 IP>:/root/offline-pkg/v2rayN.zip ~/Downloads/

# 方式二: 服务器起临时 HTTP 服务, 本地浏览器直接下载
# 服务器上 (只监听回环, 安全性最好)
cd /root/offline-pkg && python3 -m http.server 8000 --bind 127.0.0.1
# 本地另开一个终端做端口转发
ssh -fN -L 8000:127.0.0.1:8000 root@<服务器 IP>
# 然后浏览器访问 http://127.0.0.1:8000/
```

代码已另存为 [snippets/10-2.5-把文件从服务器传回来 \(拿不到软件时用这个\).sh](#) (9 行, bash)

1.3.6 2.6 如果 SSH 端口被干扰

```
# 试几个常见端口 (改服务端 sshd 的 Port, 或用临时回环 SSH)
ssh -p 443 root@<服务器 IP>
```

代码已另存为 [snippets/11-2.6-如果-SSH-端口被干扰.sh](#) (2 行, bash)

- 22 端口被 QoS 是非常常见的, 换到 2222 / 8443 / 高位随机端口通常立刻恢复。
- 服务端改端口: 编辑 /etc/ssh/sshd_config 的 Port, 先确认新端口在防火墙已放行, 再 systemctl reload ssh。
- ▲ 改端口前务必用 sshd -t 检查语法, 并用 --without-geodata 之类的无关命令验证新端口能连上, 否则会把自己锁在门外。云服务商控制台的 VNC 是你的保险。

1.4 3. 服务器基础环境准备

1.4.1 3.1 选机器

项	建议	说明
系统	Debian 12/13 或 Ubuntu 24.04	优先用 systemd 完整、内核新的版本
虚拟化	KVM / 独立机	OpenVZ 不支持，无法做 hy2 端口跳转
内存	≥ 512MB	VLESS 极省；sing-box 一体化方案留 1GB
带宽	看需求	hy2 靠带宽换速度，VPS 带宽太小体验会很差
UDP	必须支持	有些廉价 VPS 限制 UDP，不支持就别买
线路	日韩/新加坡/美西	回程线路比延迟更影响体感
IP 信誉	尽量干净	被墙过的 IP 上的协议再正确也会被封

1.4.2 3.2 SSH 加固（先活下来）

```
# 1. 先放行新端口，再改配置，最后关旧端口
NEW_SSH_PORT=42345
ufw allow ${NEW_SSH_PORT}/tcp
sed -i "s/^#\?Port .*/Port ${NEW_SSH_PORT}/" /etc/ssh/sshd_config
sshd -t && systemctl reload ssh # 注意: reload 不是 restart, 别把自己踢下线

# 2. 换到密钥登录（强烈建议）
ssh-keygen -t ed25519 -C "vps" # 在你自己电脑上执行
# 然后把公钥内容追加到服务器的 ~/.ssh/authorized_keys
# 确认密钥能登录后，再把 PasswordAuthentication 改成 no
```

代码已另存为 [snippets/12-3.2-SSH-加固（先活下来）.sh](#)（10 行，bash）

只在**确认密钥能登录**之后再关闭密码登录。顺序反了 = 永久失去这台机器（只能靠 VNC 救）。

1.4.3 3.3 系统更新与基础工具

```
apt update && apt full-upgrade -y
apt install -y curl wget ca-certificates openssl jq unzip tar \
    ufw nftables iptables chrony python3

timedatectl set-timezone Asia/Shanghai
systemctl enable --now chrony
timedatectl status # 时间必须准，否则 REALITY 握手可能失败
```

代码已另存为 [snippets/13-3.3-系统更新与基础工具.sh](#)（7 行，bash）

时间同步是 REALITY 的硬性要求。服务器时间偏差过大会导致握手直接失败。maxTimeDiff 字段默认不检查，但 TLS 本身对时间敏感。

1.4.4 3.4 开启 BBR 拥塞控制

```
cat > /etc/sysctl.d/99-bbr.conf <<'EOF'
net.core.default_qdisc = fq
net.ipv4.tcp_congestion_control = bbr
net.ipv4.tcp_fastopen = 3
# hy2 / QUIC 建议调大 UDP 缓冲
net.core.rmem_max = 16777216
net.core.wmem_max = 16777216
EOF
sysctl --system

# 确认生效
sysctl net.ipv4.tcp_congestion_control net.core.default_qdisc
```

代码已另存为 [snippets/14-3.4-开启-BBR-拥塞控制.sh](#)（12 行，bash）

hy2 主要跑 UDP/QUIC, BBR 优化的是 TCP。但服务器上通常还有 SSH、证书验证、伪装页等 TCP 服务, 所以 BBR 依然值得开。hy2 自己的拥塞控制在 bandwidth 里配 (见 §5.3)。

1.4.5 3.5 防火墙 (两道墙都要开)

```
ufw default deny incoming
ufw default allow outgoing

ufw allow 22/tcp      # 或你的新 SSH 端口
ufw allow 443/tcp     # VLESS + REALITY
ufw allow 443/udp     # Hysteria 2
ufw allow 80/tcp      # 仅 ACME HTTP-01 需要; 用 DNS-01 可不开

ufw enable
ufw status verbose
```

代码已另存为 [snippets/15-3.5-防火墙 \(两道墙都要开\).sh](#) (10 行, bash)
▲ 云服务商的安全组是独立的一层, 优先级高于服务器内的 ufw。Vultr 默认关闭所有端口, AWS/阿里云/GCP 都有安全组。ufw 开了但连不上, 九成是安全组没放行。两边都要配。

1.5 4. 方案 A: 部署 VLESS + REALITY (Xray-core)

1.5.1 4.1 关键概念 (先看懂再配)

为什么墙内首选 REALITY: 它不依赖域名, 也就不怕 DNS 污染 这是 REALITY 相对其它方案在墙内最大的实际优势, 很多人却没意识到:

你要连的东西	需不需要 DNS 解析	会不会被污染
VLESS + REALITY	不需要。客户端直接连你的 VPS IP, SNI 只是个「敲门暗号」, 不参与寻址	✓ 天然免疫
Hysteria 2	需要。客户端要先解析你的域名	▲ 可能被污染, 解析到错误 IP 就连不上

换句话说: 墙内用 REALITY 只需要「一个 IP + 一份密钥」, 不需要任何域名、DNS 记录或证书申请。这对本机的网络环境非常省心, 也少了一整类故障 (域名没配好、证书签不下来、Cloudflare 代理没关)。
而 hy2 因为强制要 TLS 证书, 就必须搞定域名 —— 这也是它部署更麻烦的原因 (见 §5.1)。

REALITY 的工作原理 REALITY 不给你自己的域名和证书, 而是在握手时借用一个真实大站的证书。当有人扫描你的 443 端口却拿不出合法的 REALITY 握手时, Xray 会把这次连接原样转发到那个大站, 于是探测者看到的是一个正常网站的证书和响应, 服务器看起来就是一个普通反向代理。

这正是对抗 GFW 主动探测的关键: GFW 会怀疑某个 IP 在偷偷转发流量, 于是主动连上去「敲一下」看反应。REALITY 让这次敲门的反应和一台真实网站服务器完全一致, 探测者就无从下手。

所以配置里有两处「借用」:

- **target** (老配置里叫 dest): 伪装转发目标, 例如 `www.microsoft.com:443`。当前版本两个字段互为 alias, 但新配置请用 **target**。
- **serverNames**: 客户端允许填的 SNI 列表, 必须包含 target 的域名。

选 target 的硬性条件 (官方原话: 目标网站最低标准是「国外网站, 支持 TLSv1.3 与 H2, 域名非跳转用」):

条件	原因
支持 TLS 1.3 + HTTP/2	REALITY 依赖 1.3 的特性; H2 保证 Server Hello 后的握手消息被加密
不是跳转域名	主域名常常 301 跳到 www, 跳转会让握手特征异常
在墙内可直连	你的 VPS 主动去连它也会失败。▲ 注意: VPS 在墙外, 所以这条是站在 VPS 的角度判断; 别选你自己墙内就连不上的站点 (如 <code>www.baidu.com</code>)
IP 离你的 VPS 越近越好	延迟低, 且看起来地理合理

先在服务器上实测一下候选目标：

```
# 换成一个你打算用的域名（在 VPS 上跑，VPS 有墙外网络）
xray tls ping www.microsoft.com:443
```

代码已另存为 [snippets/16-REALITY-的工作原理.sh](#)（2 行，bash）

看输出里是否包含 TLS 1.3、h2/ALPN h2、OCSP 等字段。

2026 年的新变化（重要）：Xray v26.3.27 起，如果选择 apple / icloud 作为 target，或使用非 443 端口，Xray 会主动输出警告——因为这两类行为在实测中极易导致服务器 IP 被封锁。所以：

- ✓ 用 443
 - ✓ target 优先选 [www.microsoft.com](#)、[learn.microsoft.com](#)、[www.cloudflare.com](#) 这类非 Apple 系大站
 - ✗ 不要用 [www.apple.com](#)（老教程里到处都是这个，现在是雷）
 - ✗ 不要为了躲 443 而换到 8443/2053 之类的端口
- ▲ 关于「偷 Cloudflare 证书」：target 最好不是 Cloudflare 那种超大 CDN 站点。原因是所有 REALITY 鉴权失败的流量都会被转发到 target，等于把你的服务器变成该站点的开放端口转发器，别人扫到就能白嫖你的流量。真要偷 Cloudflare，请配置 `limitFallbackUpload / limitFallbackDownload` 给回落连接限速。

1.5.2 4.2 安装

```
# 官方安装脚本（同时装 geoip.dat / geosite.dat）
```

```
bash -c "$(curl -L https://github.com/XTLS/Xray-install/raw/main/install-release.sh)" @ install
```

代码已另存为 [snippets/17-4.2-安装.sh](#)（2 行，bash）

安装后的文件布局（FHS 标准）：

/usr/local/bin/xray	# 可执行文件
/usr/local/etc/xray/config.json	# 主配置
/usr/local/share/xray/geoip.dat	# 后面「禁回国流量」规则要用
/usr/local/share/xray/geosite.dat	
/etc/systemd/system/xray.service	# 服务单元
/var/log/xray/	# ▲ 默认不写日志，要用 log 字段显式配置

代码已另存为 [snippets/18-4.2-安装.txt](#)（6 行，text）

常用子命令：

```
bash -c "$(curl -L https://github.com/XTLS/Xray-install/raw/main/install-release.sh)" @ help
# install --beta          装预发布版
# install -u root         以 root 身份运行（需要绑定 1024 以下端口时才用）
# install --without-geodata 不装 geo 数据（脚本已装过 geoip 可用这个更快）
# install-geodata         只更新 geoip/geosite
# remove                  卸载（保留配置和日志）
# remove --purge          彻底卸载
```

代码已另存为 [snippets/19-4.2-安装.sh](#)（7 行，bash）

确认安装：

```
xray version
systemctl status xray --no-pager
```

代码已另存为 [snippets/20-4.2-安装.sh](#)（2 行，bash）

1.5.3 4.3 生成参数

```
# UUID（客户端身份）
xray uuid
```

```
# REALITY 密钥对
xray x25519
```

代码已另存为 [snippets/21-4.3-生成参数.sh](#) (5 行, bash)

xray x25519 的输出形如:

```
PrivateKey: qwertyuiopASDFGH...=
Password (PublicKey): zxcvbnmQWERTY...=
Hash32: ...
```

代码已另存为 [snippets/22-4.3-生成参数.txt](#) (3 行, text)

这是最容易踩的坑。从 v26.3.27 起, 客户端要填的不是「Public key», 而是标着 **Password (PublicKey)** 的那一行——服务端配置里叫 `privateKey`, 客户端配置里叫 `password` (旧版叫 `publicKey`)。REALITY 的设计哲学就是「这个值对客户端而言是密码, 不该公开」, 所以官方刻意改了名。

老教程里让你把“Public key”填到客户端, 填错就会一直报握手失败。

如果私钥已经写进配置、事后才需要公钥, 可以从私钥反推:

```
xray x25519 -i "< 服务端私钥 >"
```

代码已另存为 [snippets/23-4.3-生成参数.sh](#) (1 行, bash)

`shortId` 是长度不超过 16、且长度为偶数的十六进制串 (只能 0-9a-f)。例如 `a1b2c3d4`、`a1b2` 合法, `a1b2c` 非法 (奇数长度)。留空字符串 `""` 也合法, 表示允许客户端 `shortId` 为空。

1.5.4 4.4 写配置

先建一个参数文件, 后面所有步骤共用它, 避免手工替换出错:

```
mkdir -p /root/vpn
cat > /root/vpn/params.env <<'EOF'
# ===== 基础 =====
SERVER_IP="1.2.3.4"           # ← 改: 服务器公网 IPv4
DOMAIN="hy2.example.com"     # ← 改: 你的域名 (只有 hy2 需要)
MAILTO="you@example.com"     # ← 改: ACME 注册邮箱

# ===== VLESS + REALITY =====
VLESS_PORT=443
REALITY_SNI="www.microsoft.com" # ← 改: 伪装目标域名
REALITY_TARGET="www.microsoft.com:443"
REALITY_UUID=""                # ← 填 xray uuid 的输出
REALITY_PRIVATE_KEY=""         # ← 填 xray x25519 的 PrivateKey
REALITY_PASSWORD=""           # ← 填 xray x25519 的 Password (PublicKey)
REALITY_SHORT_ID=""           # ← 填 shortId

# ===== Hysteria 2 =====
HY2_LISTEN=":443"             # 端口跳跃时改成 ":20000-20100"
HY2_AUTH=""                   # ← 填 auth 密码
HY2_OBFS=""                   # ← 填 obfs 密码
HY2_MASQ="https://www.bing.com/"
HY2_UP="100 mbps"             # VPS 上传给客户端
HY2_DOWN="500 mbps"          # 客户端传给 VPS
EOF
```

代码已另存为 [snippets/24-4.4-写配置.sh](#) (24 行, bash)

生成服务端配置 (整段当脚本跑, `bash <<'SCRIPT'` 保证 `exit` 不会关掉你的 SSH 会话):

```
bash <<'SCRIPT'
set -a; . /root/vpn/params.env; set +a

# 校验必填项, 别带着空值往下走
missing=""
for v in SERVER_IP VLESS_PORT REALITY_SNI REALITY_TARGET \
        REALITY_UUID REALITY_PRIVATE_KEY REALITY_PASSWORD REALITY_SHORT_ID; do
    eval "val=\${$v}"
    [ -n "$val" ] || missing="$missing $v"
done
if [ -n "$missing" ]; then
```

```

echo "x params.env 里这些项还是空的: $missing"
echo "    请先编辑 /root/vpn/params.env 填好再重跑本段。"
exit 1
fi

cp /usr/local/etc/xray/config.json /usr/local/etc/xray/config.json.bak 2>/dev/null

cat > /usr/local/etc/xray/config.json <<EOF
{
  "log": {
    "loglevel": "warning",
    "access": "/var/log/xray/access.log",
    "error": "/var/log/xray/error.log"
  },
  "inbounds": [
    {
      "tag": "vless-reality-in",
      "listen": "0.0.0.0",
      "port": ${VLESS_PORT},
      "protocol": "vless",
      "settings": {
        "clients": [
          {
            "id": "${REALITY_UUID}",
            "flow": "xtls-rprx-vision",
            "email": "main"
          }
        ],
        "decryption": "none"
      },
      "streamSettings": {
        "network": "raw",
        "security": "reality",
        "realitySettings": {
          "show": false,
          "target": "${REALITY_TARGET}",
          "xver": 0,
          "serverNames": ["${REALITY_SNI}"],
          "privateKey": "${REALITY_PRIVATE_KEY}",
          "shortIds": ["${REALITY_SHORT_ID}"]
        }
      },
      "sniffing": {
        "enabled": true,
        "destOverride": ["http", "tls", "quic"],
        "routeOnly": true
      }
    }
  ],
  "outbounds": [
    { "tag": "direct", "protocol": "freedom", "settings": {} },
    { "tag": "block", "protocol": "blackhole", "settings": {} }
  ],
  "routing": {
    "domainStrategy": "IPIfNonMatch",
    "rules": [
      { "type": "field", "ip": ["geoip:private"], "outboundTag": "block" },
      { "type": "field", "protocol": ["bittorrent"], "outboundTag": "block" },
      { "type": "field", "ip": ["geoip:cn"], "outboundTag": "block" }
    ]
  }
}
EOF

```

```

}
}
EOF

echo "✓ 已写入 /usr/local/etc/xray/config.json"
SCRIPT

```

代码已另存为 [snippets/25-4.4-写配置.sh](#) (77 行, bash)

逐项解释那些容易被抄错的字段:

字段	值	说明
settings.clients	数组	VLESS 入站的用户列表是 clients (不是 users, users 是出站的写法)
flow	xtls-rprx-vision	XTLS Vision 流控。用了 REALITY 就必须用 Vision, 这是抗 TLS-in-TLS 特征的核心
decryption	none	VLESS 自身不加密, 加密由 REALITY 负责
streamSettings.network	raw	当前官方示例用的值 (旧名 tcp, 仍兼容)。REALITY 只支持 raw/xhttp/grpc
target	域名:443	旧名 dest, 两字段互为 alias。核心判断依据是「这个字段存在与否」来区分服务端/客户端, 写错位置会导致识别异常
privateKey	x25519 私钥	只放在服务端 。泄露 = 别人能伪装成你的服务器
shortIds	数组	可配多个, 方便以后给不同客户端发不同的 id
sniffing	开启	为了让下面的 protocol: bittorrent 规则能生效
routing 里 geoip:cn → block	—	禁回国流量 。官方明确把这条列为「配置加分项」: 服务器主动连国内 IP 是非常明显的特征

为什么一定要禁回国流量、且只配 Vision: 官方给出的「正确配置」四条是——(1) 服务端用合理端口并**禁回国流量**; (2) 只配 XTLS Vision, 不要和普通 TLS 代理混用; (3) 回落是**网页**, 不要回落到其它代理协议; (4) 客户端**必须**开 uTLS 指纹。违反这四条, Vision 的抗封优势基本归零。

1.5.5 4.5 校验并启动

```

# 语法检查 (** 永远先做这一步 **)
xray run -test -config /usr/local/etc/xray/config.json

systemctl enable --now xray
systemctl restart xray
systemctl status xray --no-pager
journalctl -u xray -n 50 --no-pager

# 确认端口真的在监听, 且只被 xray 占
ss -tulpen | grep -E ':443[[:space:]]'

```

代码已另存为 [snippets/26-4.5-校验并启动.sh](#) (10 行, bash)

看到 Xray 26.x.x ... started 且无 error 就成了。

1.5.6 4.6 顺手备一份参数记录

```

set -a; . /root/vpn/params.env; set +a

cat > /root/vpn/creds.txt <<EOF
# 私人文件, chmod 600, ** 不要发给任何人 **
VLESS 地址      : ${SERVER_IP}
VLESS 端口      : ${VLESS_PORT}
UUID            : ${REALITY_UUID}
Flow            : xtls-rprx-vision
传输            : raw
安全            : reality
SNI             : ${REALITY_SNI}

```

```
公钥 (pbk/password) : ${REALITY_PASSWORD}
ShortId(sid) : ${REALITY_SHORT_ID}
Fingerprint : chrome
EOF
chmod 600 /root/vpn/creds.txt
cat /root/vpn/creds.txt
```

代码已另存为 [snippets/27-4.6-顺手备一份参数记录.sh](#) (17 行, bash)

1.6 5. 方案 B: 部署 Hysteria 2

1.6.1 5.1 它和 VLESS 的根本区别

Hysteria 2 跑在 QUIC (UDP) 上, 强制要求 TLS 证书。所以你必须二选一:

证书方案	需要域名	客户端设置	抗封锁	建议
ACME 自动签发	✓ 需要	insecure: false + SNI 填域名	强	✓ 有域名就选这个
自签证书	✗ 不需要	insecure: true, 或用 pinSHA256 固定指纹	中	域名不是你的、或证书签不下来时用

另外: 如果你用 Cloudflare 托管域名, 必须设成 DNS only (灰云)。开了橙云的话, 客户端连的是 Cloudflare 而不是你的 VPS, 而 Cloudflare 的常规代理不转发 Hysteria2 的 UDP 流量, 结果就是连不上。

A hy2 → <你的VPS_IP> TTL=Auto 代理状态: DNS only (灰云)! 不要橙云

代码已另存为 [snippets/28-5.1-它和-VLESS-的根本区别.txt](#) (1 行, text)

1.6.2 5.2 安装

```
# 官方安装脚本 (会自动装 systemd 服务并写一份示例配置)
bash <(curl -fsSL https://get.hy2.sh/)
```

代码已另存为 [snippets/29-5.2-安装.sh](#) (2 行, bash)

脚本干了什么、没干什么, 搞清楚很重要 (很多教程说错了):

- ✓ 装二进制到 /usr/local/bin/hysteria
- ✓ 装 hysteria-server.service 和 hysteria-server@.service
- ✓ 建 hysteria 用户、/etc/hysteria/ 目录
- ✓ 仅当配置文件不存在时写入一份示例 config.yaml (里面带随机密码、域名是 your.domain.net 的占位符)
- ✗ 不会帮你申请证书 (acme 段是占位域名, 你必须自己改)
- ✗ 不会生成客户端配置

常用参数:

```
bash <(curl -fsSL https://get.hy2.sh/) -- --check # 检查更新
bash <(curl -fsSL https://get.hy2.sh/) -- --version v2.8.2
bash <(curl -fsSL https://get.hy2.sh/) -- --force # 强制重装
bash <(curl -fsSL https://get.hy2.sh/) -- --remove # 卸载
bash <(curl -fsSL https://get.hy2.sh/) -- --local ./hysteria # 用本地已有的二进制安装 (离线场景用)
```

代码已另存为 [snippets/30-5.2-安装.sh](#) (5 行, bash)

服务的关键配置 (从官方安装脚本源码可确认):

```
# /etc/systemd/system/hysteria-server.service
ExecStart=/usr/local/bin/hysteria server --config /etc/hysteria/config.yaml
User=hysteria
AmbientCapabilities=CAP_NET_ADMIN CAP_NET_BIND_SERVICE CAP_NET_RAW
```

代码已另存为 [snippets/31-5.2-安装.ini](#) (4 行, ini)

这几个 capability 是有意给的: CAP_NET_BIND_SERVICE 让你不用 root 也能绑 443; CAP_NET_ADMIN 是端口跳跃要改防火墙规则。所以 hy2 完全可以在非 root 下跑。

1.6.3 5.3 配置要点逐项说明

方案 1: ACME 自动签发 (推荐)

```
# /etc/hysteria/config.yaml
# 监听地址。省略则默认 :443。只写端口号 = 同时监听 IPv4+ IPv6;
# 只想要 IPv4 就写 0.0.0.0:443
listen: :443

acme:
  domains:
    - hy2.example.com
  email: you@example.com
  # type: http # 默认 http (HTTP-01, 需要 80 端口通)
  # type: dns # 改用 DNS-01, 不需要 80 端口, 但要配 DNS provider 凭据

auth:
  type: password
  password: "换成强密码"

obfs:
  # Salamander 混淆: 把 QUIC 包打乱成看似随机的字节流
  type: salamander
  salamander:
    password: "换成另一个强密码"

masquerade:
  # 有人扫端口/发非 hy2 请求时, 反代到这个正常网站
  type: proxy
  proxy:
    url: https://www.bing.com/
    rewriteHost: true

bandwidth:
  # 不填则用 BBR 拥塞控制
  up: 100 mbps # VPS → 客户端
  down: 500 mbps # 客户端 → VPS
```

代码已另存为 `snippets/32-5.3-配置要点逐项说明.yaml` (30 行, yaml)

▲ bandwidth 的方向极易填反。up 是服务器发出的速率 (对应你下载), down 是服务器接收的速率 (对应你上传)。

另一个反直觉的点: 填了 bandwidth 就用 Hysteria 自己的 Brutal 拥塞控制 (主动按你给的速率猛发包), 不填则退回 BBR (更温和)。Brutal 在网络好时更快, 但更容易触发运营商 QoS 被限速。所以:

- 网络质量好、追求极速 → 填 bandwidth, 数值略低于你的实际带宽
- 容易被限速、或不确定 → 留空让它用 BBR, 更稳

方案 2: 自签证书 (无域名)

```
mkdir -p /etc/hysteria
# -pkeyopt ec_paramgen_curve:P-256 直接生成 P-256 曲线 (OpenSSL 1.1.1+ / 3.x 均可)
# 不要用 ec:<(openssl ecparam ...) 这种进程替换写法 —— 它依赖 bash, 且在部分发行版上会失败
openssl req -x509 -nodes -newkey ec -pkeyopt ec_paramgen_curve:P-256 \
  -keyout /etc/hysteria/server.key \
  -out /etc/hysteria/server.crt \
  -subj "/CN=bing.com" \
  -addext "subjectAltName=DNS:bing.com" \
  -days 36500
chown -R hysteria:hysteria /etc/hysteria
chmod 600 /etc/hysteria/server.key

# 确认 SAN 真的写进去了 (现代 TLS 客户端只看 SAN, 不认 CN)
openssl x509 -in /etc/hysteria/server.crt -noout -text | grep -A1 "Alternative"
```

代码已另存为 `snippets/33-5.3-配置要点逐项说明.sh` (14 行, bash)

```
listen: :443
```

```
tls: # 注意: tls 和 acme 二选一, 不能同时出现
cert: /etc/hysteria/server.crt
key: /etc/hysteria/server.key
# sniGuard: strict # 严格校验客户端 SNI, 默认 dns-san
```

代码已另存为 [snippets/34-5.3-配置要点逐项说明.yaml](#) (6 行, yaml)

```
# 记下证书指纹, 客户端用它替代 insecure (比关校验安全得多)
openssl x509 -in /etc/hysteria/server.crt -outform der | \
openssl dgst -sha256 -binary | openssl enc -base64
```

代码已另存为 [snippets/35-5.3-配置要点逐项说明.sh](#) (3 行, bash)

客户端填 pinSHA256: < 上面那串 > 即可, 无需关闭证书校验。

用 EC 证书 (上面的 P-256) 而不是 Ed25519: sing-box 1.14+ 的客户端会模拟 Chrome 的 QUIC 握手指纹, 而 Chrome 不宣告支持 Ed25519, 服务端用 Ed25519 证书会直接握手失败。ECDSA 或 RSA 证书都安全。

端口跳跃 (应对单端口被 QoS)

```
# /etc/hysteria/config.yaml ——服务端, 只要这一行
listen: :20000-20100 # 监听一个范围
```

代码已另存为 [snippets/36-5.3-配置要点逐项说明.yaml](#) (2 行, yaml)

原理 (Linux 内置支持): 服务端只真正绑定范围内的第一个端口, 再自动用 nftables 或 iptables 建立规则, 把范围内其他端口的入站流量重定向到第一个端口; 进程退出时规则自动清理。

前提: 系统里要有 nft (nftables) 或 iptables/ip6tables, 且进程需要 root 或 CAP_NET_ADMIN 才能改防火墙规则。官方安装脚本给的 systemd 单元已经带了 AmbientCapabilities=CAP_NET_ADMIN, 正常装完就有。

```
# 防火墙必须放行整个范围 (ufw 的范围写法是 起始: 结束)
ufw allow 20000:20100/udp
# 云服务商安全组也要放行 20000-20100/udp
```

代码已另存为 [snippets/37-5.3-配置要点逐项说明.sh](#) (3 行, bash)

客户端要指定端口范围, 并可设置跳换间隔 (transport.udp 段):

```
transport:
  udp:
    hopInterval: 30s # 固定间隔, 默认就是 30s, 最小 5s
```

代码已另存为 [snippets/38-5.3-配置要点逐项说明.yaml](#) (3 行, yaml)

想要随机间隔 (更难被预测和识别) 就把上面两行换成:

```
transport:
  udp:
    minHopInterval: 15s # 最小间隔, 最小 5s
    maxHopInterval: 45s # 最大间隔
```

代码已另存为 [snippets/39-5.3-配置要点逐项说明.yaml](#) (4 行, yaml)

▲ hopInterval 和 minHopInterval/maxHopInterval 二选一, 不能同时存在。

客户端也支持更灵活的混合写法 (多个单端口 + 范围的组合, 端口数不限): example.com:1234,5000-6000,7044

sing-box 里对应 server_ports: ["20000:20100"] (见 §6.2)。

要不要开? 端口跳跃不是越多越好:

代价	说明
防火墙规则数量	范围越大规则越多, 部分 VPS 的安全组有规则数上限 (比如 200 条)
内核要求	需要 nftables 或 iptables, 以及相应的 NAT 模块; 精简内核可能没编译
客户端支持	sing-box / Shadowrocket / Mihomo 支持; v2rayNG、部分 iOS 客户端不支持

代价	说明
收益	只在「单端口被限速，换个端口就恢复」时才有意义

443/UDP 稳定就别开。开了之后连不上，**第一件事是恢复成单端口**（别一上来就去排查网络）。

1.6.4 5.4 生成配置并启动

```
bash <<'SCRIPT'
set -a; . /root/vpn/params.env; set +a

# 生成两个强随机密码（用 hex，不用 base64：
# base64 会产生 / + = 三种字符，/ 会截断路径、+ 在 URI 里会被解成空格，写 YAML/URI 都很容易出坑）
HY2_AUTH=$(openssl rand -hex 24)
HY2_OBFS=$(openssl rand -hex 24)
echo "auth 密码: $HY2_AUTH"
echo "obfs 密码: $HY2_OBFS"

cp /etc/hysteria/config.yaml /etc/hysteria/config.yaml.bak 2>/dev/null

cat > /etc/hysteria/config.yaml <<EOF
listen: ${HY2_LISTEN}

acme:
  domains:
    - ${DOMAIN}
  email: ${MAILTO}

auth:
  type: password
  password: "${HY2_AUTH}"

obfs:
  type: salamander
  salamander:
    password: "${HY2_OBFS}"

masquerade:
  type: proxy
  proxy:
    url: ${HY2_MASQ}
    rewriteHost: true

bandwidth:
  up: ${HY2_UP}
  down: ${HY2_DOWN}
EOF

# 把密码回写进 params.env，后面生成客户端配置要用
sed -i "s|^HY2_AUTH=.*|HY2_AUTH=\"${HY2_AUTH}\"|" /root/vpn/params.env
sed -i "s|^HY2_OBFS=.*|HY2_OBFS=\"${HY2_OBFS}\"|" /root/vpn/params.env

systemctl enable --now hysteria-server
systemctl restart hysteria-server
systemctl status hysteria-server --no-pager
journalctl -u hysteria-server -n 50 --no-pager

ss -ulpen | grep -E ':443[[:space:]]'
SCRIPT
```

代码已另存为 [snippets/40-5.4-生成配置并启动.sh](#) (51 行, bash)

看到 server up and running 就成了。

首次启动会自动去申请证书。如果失败，按顺序检查：

1. 域名是否已解析到这台 VPS: `dig +short ${DOMAIN}`
2. Cloudflare 是否为灰云
3. 80/tcp 是否放行 (ufw + 安全组)
4. `journalctl -u hysteria-server -e` 看具体报错
5. Let's Encrypt 有频率限制，失败后等一小时再试；或改用 DNS-01 / 自签证书

1.7 5.5 附录：场景 B ----- 服务器无外网时的离线安装（你的情况用不到）

如果你的服务器真的连不上 GitHub（内网机器、限制出网的实例），在任意一台能上网的机器上准备好安装包，然后传进去。

1.7.1 准备离线包（在能上网的机器上执行）

```
mkdir -p offline-pkg && cd offline-pkg

# Xray: zip 本身就含 xray 二进制 + geoip.dat + geosite.dat
XVER=v26.3.27
curl -fLO "https://github.com/XTLS/Xray-core/releases/download/${XVER}/Xray-linux-64.zip"
curl -fLO "https://github.com/XTLS/Xray-core/releases/download/${XVER}/Xray-linux-64.zip.dgst"

# Hysteria 2 (文件名格式: hysteria-linux-amd64, 无扩展名)
curl -fLO
  ↪ "https://github.com/apernet/hysteria/releases/download/app/v2.8.2/hysteria-linux-amd64"
chmod +x hysteria-linux-amd64

# 客户端也一并准备好（第三节要用）
curl -fLO "https://github.com/2dust/v2rayN/releases/latest/download/v2rayN-With-Core.zip"

# 校验（务必做）
sha256sum -c Xray-linux-64.zip.dgst
ls -lh
```

代码已另存为 [snippets/41-准备离线包（在能上网的机器上执行）.sh](#)（17 行，bash）

版本号请以实际发布页为准；不确定就先进 GitHub Releases 页面看一眼再改。

1.7.2 传进服务器

```
# 方式一：scp
scp offline-pkg/* root@<服务器 IP>:/root/offline-pkg/

# 方式二：服务器完全没有网络时用 U 盘 / 内网共享 / 私有对象存储
```

代码已另存为 [snippets/42-传进服务器.sh](#)（4 行，bash）

1.7.3 在无外网的服务器上安装

Xray（手工装，包含 systemd 单元）：

```
apt install -y unzip # 如果 apt 也不可用，用 busybox 或自带解压工具
cd /root/offline-pkg
unzip -o Xray-linux-64.zip
install -m 755 xray /usr/local/bin/xray
mkdir -p /usr/local/share/xray /usr/local/etc/xray /var/log/xray
install -m 644 geoip.dat geosite.dat /usr/local/share/xray/
chmod 755 geoip.dat geosite.dat

cat > /etc/systemd/system/xray.service <<'EOF'
[Unit]
Description=Xray Service
Documentation=https://github.com/XTLS/Xray-core
After=network.target nss-lookup.target
```

```

[Service]
User=nobody
CapabilityBoundingSet=CAP_NET_ADMIN CAP_NET_BIND_SERVICE
AmbientCapabilities=CAP_NET_ADMIN CAP_NET_BIND_SERVICE
NoNewPrivileges=true
ExecStart=/usr/local/bin/xray run -config /usr/local/etc/xray/config.json
Restart=on-failure
RestartPreventExitStatus=23
LimitNPROC=10000
LimitNOFILE=1000000

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now xray
xray version

```

代码已另存为 [snippets/43-在无外网的服务器上安装.sh](#) (32 行, bash)

Hysteria 2 (用官方的 `--local` 参数, 逻辑和你在有网时完全一样):

```

cd /root/offline-pkg
# 顺便把 systemd 单元一起装上
curl -fsSL https://get.hy2.sh/ 2>/dev/null > /dev/null || true # 无网时跳过, 改用手写单元
bash install_server.sh --local ./hysteria-linux-amd64

```

代码已另存为 [snippets/44-在无外网的服务器上安装.sh](#) (4 行, bash)

如果连 `install_server.sh` 都下不到, 手写单元即可 (内容与官方一致):

```

install -Dm755 hysteria-linux-amd64 /usr/local/bin/hysteria
id -u hysteria >/dev/null 2>&1 || useradd -r -d /var/lib/hysteria -m hysteria
mkdir -p /etc/hysteria

cat > /etc/systemd/system/hysteria-server.service <<'EOF'
[Unit]
Description=Hysteria Server Service
After=network.target

[Service]
Type=simple
ExecStart=/usr/local/bin/hysteria server --config /etc/hysteria/config.yaml
WorkingDirectory=~
User=hysteria
Group=hysteria
Environment=HYSTERIA_LOG_LEVEL=info
CapabilityBoundingSet=CAP_NET_ADMIN CAP_NET_BIND_SERVICE CAP_NET_RAW
AmbientCapabilities=CAP_NET_ADMIN CAP_NET_BIND_SERVICE CAP_NET_RAW
NoNewPrivileges=true

[Install]
WantedBy=multi-user.target
EOF

systemctl daemon-reload
systemctl enable --now hysteria-server

```

代码已另存为 [snippets/45-在无外网的服务器上安装.sh](#) (26 行, bash)

1.7.4 ▲ 无外网环境下的证书问题

ACME 自动签发必须有出网能力。所以：

- 用自签证书（见 §5.3 配置要点 里的「方案 2：自签证书」），客户端配 `insecure: true` 或 `pinSHA256`。
- 如果你有域名但服务器无外网：在家宽/另一台机器上用 `certbot` 或 `acme.sh` 申请好证书，把 `fullchain.pem` + `privkey.pem` 一起传进服务器，然后配置：

```
tls:
  cert: /etc/hysteria/fullchain.pem
  key: /etc/hysteria/privkey.pem
```

代码已另存为 [snippets/46-▲-无外网环境下的证书问题.yaml](#)（3 行，yaml）

注意 Let's Encrypt 证书 90 天到期，**离线环境要提前准备续期**（把新证书带进去替换 + `systemctl restart hysteria-server`）。

1.8 6. 客户端配置

1.8.1 6.1 生成所有格式的客户端配置（在服务器上一键产出）

在服务器上跑这段，直接把输出复制走：

```
set -a; source /root/vpn/params.env; set +a

echo "=====
echo " 1) VLESS+ REALITY 分享链接 (v2rayN/v2rayNG/Shadowrocket 都能扫) "
echo "=====
cat <<EOF
vless://${REALITY_UUID}@${SERVER_IP}:${VLESS_PORT}?encryption=none&flow=xtls-rprx-vision&security=
  ty=reality&sni=${REALITY_SNI}&fp=chrome&pbk=${REALITY_PASSWORD}&sid=${REALITY_SHORT_ID}&typ
  e=tcp&headerType=none#REALITY-${SERVER_IP}
EOF

echo
echo "=====
echo " 2) Hysteria2 分享链接"
echo "=====
cat <<EOF
hysteria2://${HY2_AUTH}@${DOMAIN}:443?sni=${DOMAIN}&insecure=0&obfs=salamander&obfs-password=${
  HY2_OBFS}#HY2-${DOMAIN}
EOF

echo
echo "=====
echo " 3) sing-box 客户端 outbound (VLESS + REALITY) "
echo "=====
cat <<EOF
{
  "type": "vless",
  "tag": "proxy-reality",
  "server": "${SERVER_IP}",
  "server_port": ${VLESS_PORT},
  "uuid": "${REALITY_UUID}",
  "flow": "xtls-rprx-vision",
  "packet_encoding": "xudp",
  "tls": {
    "enabled": true,
    "server_name": "${REALITY_SNI}",
    "utls": { "enabled": true, "fingerprint": "chrome" },
    "reality": {
      "enabled": true,
      "public_key": "${REALITY_PASSWORD}",
      "short_id": "${REALITY_SHORT_ID}"
    }
  }
}
```

```

    }
  }
}
EOF

echo
echo "=====
echo " 4) sing-box 客户端 outbound (Hysteria2) "
echo "=====
cat <<EOF
{
  "type": "hysteria2",
  "tag": "proxy-hy2",
  "server": "${DOMAIN}",
  "server_port": 443,
  "password": "${HY2_AUTH}",
  "obfs": { "type": "salamander", "password": "${HY2_OBFS}" },
  "up_mbps": 100,
  "down_mbps": 500,
  "tls": { "enabled": true, "server_name": "${DOMAIN}", "insecure": false }
}
EOF

```

代码已另存为 [snippets/47-6.1-生成所有格式的客户端配置](#) (在服务器上一键产出) .sh (60 行, bash)

1.8.2 6.2 sing-box 完整客户端配置 (无图形界面时用这个)

一台机器同时挂两个节点, 并做自动故障转移:

```

{
  "log": { "level": "info", "timestamp": true },
  "inbounds": [
    { "type": "mixed", "tag": "mixed-in", "listen": "127.0.0.1", "listen_port": 2080 }
  ],
  "outbounds": [
    {
      "type": "vless", "tag": "proxy-reality",
      "server": "1.2.3.4", "server_port": 443,
      "uuid": "<REALITY_UUID>", "flow": "xtls-rprx-vision",
      "tls": {
        "enabled": true, "server_name": "www.microsoft.com",
        "utls": { "enabled": true, "fingerprint": "chrome" },
        "reality": { "enabled": true, "public_key": "<REALITY_PASSWORD>", "short_id":
          ↪ "<REALITY_SHORT_ID>" }
      }
    },
    {
      "type": "hysteria2", "tag": "proxy-hy2",
      "server": "hy2.example.com", "server_port": 443,
      "password": "<HY2_AUTH>",
      "obfs": { "type": "salamander", "password": "<HY2_OBFS>" },
      "up_mbps": 100, "down_mbps": 500,
      "tls": { "enabled": true, "server_name": "hy2.example.com" }
    },
    { "type": "urltest", "tag": "auto", "outbounds": ["proxy-reality", "proxy-hy2"], "url":
      ↪ "https://www.gstatic.com/generate_204", "interval": "3m", "tolerance": 50 },
    { "type": "direct", "tag": "direct" }
  ],
  "route": {
    "rules": [
      { "ip_is_private": true, "outbound": "direct" }
    ]
  }
}

```

```

    "final": "auto"
  }
}

```

代码已另存为 `snippets/48-6.2-sing-box-完整客户端配置（无图形界面时用这个）.json`（34 行, json）

```

sing-box check -c config.json    # 校验，必须先过这一步
sing-box run -c config.json      # 前台跑，看日志

```

代码已另存为 `snippets/49-6.2-sing-box-完整客户端配置（无图形界面时用这个）.sh`（2 行, bash）

- 想接管全局流量（而不是只有 2080 端口的 SOCKS），加一个 TUN 进站：

```

{ "type": "tun", "tag": "tun-in", "address": ["172.19.0.1/30", "fdfe:dcba:9876::1/126"],
  ↪ "auto_route": true, "strict_route": true, "stack": "mixed" }

```

代码已另存为 `snippets/50-6.2-sing-box-完整客户端配置（无图形界面时用这个）.json`（1 行, json）

Linux/macOS 需要 root/管理员权限；Windows 需要管理员权限。字段名随版本变动，以 `sing-box check` 的报错为准。

- sing-box 的 hy2 端口跳跃字段（1.11.0+）：把 `server_port` 换成

```

"server_ports": ["20000:20100"],
"hop_interval": "30s",
"hop_interval_max": "45s"

```

代码已另存为 `snippets/51-6.2-sing-box-完整客户端配置（无图形界面时用这个）.json`（3 行, json）

（`hop_interval_max` 是 1.14.0+，用于在 min/max 之间随机跳，更难被预测。）

- ▲ sing-box 1.14 起内联 `acme` 字段已废弃，改用 `certificate_provider`（acme 内联写法将在 1.16 移除）。用 sing-box 签证书时注意版本。

1.8.3 6.3 Mihomo / Clash Verge Rev / FIClash (yaml)

proxies:

```

- name: "RE-REALITY"
  type: vless
  server: 1.2.3.4
  port: 443
  uuid: "<REALITY_UUID>"
  network: tcp
  udp: true
  tls: true
  flow: xtls-rprx-vision
  servername: www.microsoft.com
  client-fingerprint: chrome
  reality-opts:
    public-key: "<REALITY_PASSWORD>"
    short-id: "<REALITY_SHORT_ID>"

- name: "HY2"
  type: hysteria2
  server: hy2.example.com
  port: 443
  # ports: 20000-20100          # 端口跳跃，需要服务端也开了才生效
  password: "<HY2_AUTH>"
  obfs: salamander
  obfs-password: "<HY2_OBFS>"
  sni: hy2.example.com
  skip-cert-verify: false     # 自签证书才改 true
  up: "100 Mbps"
  down: "500 Mbps"

```

```

proxy-groups:
- name: " 自动选择"
  type: url-test
  proxies: ["RE-REALITY", "HY2"]
  url: "https://www.gstatic.com/generate_204"
  interval: 300
  tolerance: 50

rules:
- GEOIP,CN,DIRECT
- GEOSITE,category-ads-all,REJECT
- MATCH, 自动选择

```

代码已另存为 [snippets/52-6.3-Mihomo-Clash-Verge-Rev-FlClash\(yaml\).yaml](#) (41 行, yaml)
各客户端的 yaml 字段名会随版本变动 (Mihomo 系尤其快)。优先用[分享链接导入](#)，导入失败再手写 yaml，并以该项目当期文档为准。

1.8.4 6.4 客户端选择

平台	推荐	支持 HY2	端口跳跃
Windows	v2rayN、Furious	✓	✓
macOS	Happ、Streisand、OneXray、v2rayN、v2rayA	✓	✓
Linux	v2rayA (Web 界面)、sing-box、mihomo	✓	✓
iOS / iPadOS / tvOS	Shadowrocket、Happ、Streisand、Loon、Egern、Quantumult X	✓	✓
Android	v2rayNG、Karing、Hiddify、NekoBox、SagerNet 官方 App	✓	部分
路由器 / OpenWrt	PassWall 2、luci-app-xray	✓	✓

想「一个客户端搞定两种协议」，sing-box 是最省心的选择（它同时也是服务端方案，见 §8）。

1.9 7. 拿到客户端软件：墙内这步是真实阻塞点

服务器上的活儿全干完之后，最后一个障碍是**你本机**：GFW 让 GitHub Releases 打不开，v2rayN / v2rayNG / Shadowrocket 这类客户端装不上。

好消息是：这一节的手段**全部依赖你已经打通的 SSH 通道**，不需要额外任何条件。按顺序试，通常方法 1 或 2 就够了。

1.9.1 方法 1：用 SSH SOCKS5 当下载代理（最通用）

先按 §0.1 / §2.1 开好 SOCKS5，然后让安装包管理器走代理：

```

export ALL_PROXY=socks5h://127.0.0.1:1080

# 先验证代理对墙外站点确实生效
curl -s https://api.ipify.org; echo          # 应返回 VPS 的 IP

# 然后正常下载 (git / wget / curl / npm / pip 都认 ALL_PROXY)
curl -fLO https://github.com/2dust/v2rayN/releases/latest/download/v2rayN-With-Core.zip

```

代码已另存为 [snippets/53-方法-1：用-SSH-SOCKS5-当下载代理（最通用）.sh](#) (7 行, bash)
如果 curl 走代理还是超时，多半是 ALL_PROXY 没被某些工具读取 —— 那种情况直接用 curl -x socks5h://127.0.0.1:1080 <url> 显式指定（见 §2.3）。

1.9.2 方法 2：从服务器反向推给你（最稳，推荐）

让 VPS 帮你下，你只负责 scp 拉回来。这条路不依赖本机任何代理设置，SSH 能连就一定成：

```

# ---- 在服务器上执行（它有墙外网络，随便下） ----
mkdir -p /root/offline-pkg/clients && cd /root/offline-pkg/clients
curl -fLO "https://github.com/2dust/v2rayN/releases/latest/download/v2rayN-With-Core.zip"
ls -lh      # 确认文件大小正常，不是 0 字节或错误页

```

代码已另存为 [snippets/54-方法-2：从服务器反向推给你（最稳，推荐）.sh](#) (4 行, bash)

```
# ---- 在你本机执行 ----
```

```
scp root@<服务器 IP>:/root/offline-pkg/clients/v2rayN-With-Core.zip ~/Downloads/
```

代码已另存为 [snippets/55-方法-2: 从服务器反向推给你 \(最稳, 推荐\).sh](#) (2 行, bash)

Windows 用户注意: 从 Linux 服务器 scp 下来的可执行文件没有 Windows 权限/签名, 可能被 SmartScreen 或杀软拦截。解法是下载官方的 Windows 压缩包 (自带.exe), 传过来后先「属性 → 解除锁定」, 再考虑加白名单。

手机端同理: iOS 上装 Shadowrocket 需要境外 App Store 账号 (切地区后免费下载); Android 上 v2rayNG 的 apk 可以用同样方式从服务器 scp 回来。

1.9.3 方法 3: 临时用浏览器顶一阵

在客户端装好之前, ssh -D 1080 + Firefox 的 SOCKS5 设置已经足够处理绝大多数上网需求 (浏览、看文档、下载小文件)。别小看它——很多人在正式节点部署好之前, 就是靠这个过渡的, 而且它没有任何被检测的特征 (就是普通 SSH)。

▲ 只适合临时应急。SSH 隧道的缺点: 速度受跨境链路限制、只能给本机浏览器用、给不了手机/其他 App、22 端口长期使用特征明显。正式节点配好后, 就别再靠它上网了。

1.9.4 方法 4: 手机上

手机没法直接开 SSH 隧道 (除非你装了 SSH 客户端 App), 两条路:

1. 在电脑上开 ssh -D 1080, 手机和电脑连同一 Wi-Fi, 手机上配 SOCKS5 指向 电脑局域网 IP:1080 (电脑防火墙放行 1080, 仅限局域网);
2. 手机上装个 SSH 客户端 (iOS: Termius / Blink; Android: Termius / JuiceSSH), 用它的本地端口转发功能开 SOCKS5, 再让浏览器指向它。

Android 上 Termux 还可以 pkg add sing-box 直接原生运行 (注意: Termux 构建的二进制在部分场景下有兼容问题, 见 sing-box 官方文档的「Problematic Sources」)。

1.9.5 关于各类 GitHub 镜像站

网上 lots of 「GitHub 加速镜像」教程。这里不给推荐, 原因:

- 绝大多数已失效或不稳定, 且镜像站能看到你下载的每一个字节, 包括你的 UUID、密钥、订阅链接;
- 代理协议被封后, 最先坏掉的就是这些依赖境外 DNS 和境外带宽的中转服务。

镜像只用来加速「跟代理无关的东西」(pip/npm/系统更新), 绝不用于下载代理软件本身。代理软件必须从官方源或官方 CDN 拿, 验证 GPG/sha256。

1.10 8. 进阶: 一个 sing-box 进程同时跑两种协议

sing-box 可以同时监听 TCP 443 (VLESS+REALITY) 和 UDP 443 (Hysteria2) ——TCP 和 UDP 是独立端口空间, 不冲突。好处是:

- 只有一个二进制、一个配置文件、一个 systemd 服务;
- 内存占用比「Xray + Hysteria2」两个进程更低;
- 一套工具同时是服务端和客户端。

```
# 安装
```

```
curl -fsSL https://sing-box.app/install.sh | sh
```

```
# 或 Debian/Ubuntu 用官方源
```

```
# sudo mkdir -p /etc/apt/keyrings
```

```
# sudo curl -fsSL https://sing-box.app/gpg.key -o /etc/apt/keyrings/sagernet.asc
```

```
# echo 'Types: deb\nURIs: https://deb.sagernet.org/\nSuites: *\nComponents: *\nEnabled:
```

```
yes\nSigned-By: /etc/apt/keyrings/sagernet.asc' | sudo tee
```

```
/etc/apt/sources.list.d/sagernet.sources
```

```
# sudo apt-get update && sudo apt-get install sing-box
```

```
sing-box version
```

```
sing-box generate reality-keypair # 生成 REALITY 密钥对
```

```
sing-box check -c /etc/sing-box/config.json # 校验配置
```

```
systemctl enable --now sing-box
```

代码已另存为 [snippets/56-8.-进阶: 一个-sing-box-进程同时跑两种协议.sh](#) (12 行, bash)

服务端配置骨架:

```

{
  "log": { "level": "info", "timestamp": true },
  "inbounds": [
    {
      "type": "vless", "tag": "in-reality", "listen": "0.0.0.0", "listen_port": 443,
      "users": [{ "uuid": "<REALITY_UUID>", "name": "main" }],
      "tls": {
        "enabled": true,
        "server_name": "www.microsoft.com",
        "reality": {
          "enabled": true,
          "handshake": { "server": "www.microsoft.com", "server_port": 443 },
          "private_key": "<REALITY_PRIVATE_KEY>",
          "short_id": ["<REALITY_SHORT_ID>"]
        }
      }
    },
    {
      "type": "hysteria2", "tag": "in-hy2", "listen": "0.0.0.0", "listen_port": 443,
      "users": [{ "password": "<HY2_AUTH>" }],
      "obfs": { "type": "salamander", "password": "<HY2_OBFS>" },
      "up_mbps": 100, "down_mbps": 500,
      "tls": { "enabled": true, "server_name": "hy2.example.com", "certificate_path":
        ↪ "/etc/ssl/hy2/fullchain.pem", "key_path": "/etc/ssl/hy2/privkey.pem" }
    }
  ],
  "outbounds": [ { "type": "direct", "tag": "direct" } ]
}

```

代码已另存为 `snippets/57-8.-进阶：一个sing-box-进程同时跑两种协议.json` (27 行, json)

要点：

- sing-box 的 REALITY 服务端用 `tls.reality.handshake` 指定伪装目标, `private_key` / `short_id` 在 `tls.reality` 下;
 - sing-box 不内置「回落转发」那套行为, 实现上和 Xray 略有差异, 实测抗封锁表现不如 Xray 成熟。追求极致隐蔽性就用 Xray;
 - hy2 证书这里用的是文件路径。sing-box 1.14+ 想让它自动签证书, 得用 `certificate_provider` (内联 acme 已废弃)。但 hy2 端口跳跃方面 sing-box 客户端很强, 服务端则建议继续用官方 Hysteria 2;
 - 首次上线务必 sing-box `check -c`, sing-box 对未知字段会直接报错, 能帮你避开大部分版本差异问题。
- 另外补一个 2026 年的新选项: Xray-core v26.3.27 已内置 Hysteria 2 入站与传输层。也就是说理论上 Xray 也能一个进程跑两种协议。但官方明确说明: 用 Xray 做端口跳跃时, 入站只监听一个端口, 其他端口用 iptables 转发, 不要让入站直接监听范围。除非你很熟悉 Xray 的 `Finalmask` / `quicParams` 新参数, 否则优先用官方 Hysteria 2 跑 hy2。

1.11 9. 验收清单

复制到服务器上执行, 逐项确认:

```

cat > /root/vpn/check.sh <<'SH'
#!/usr/bin/env bash
set -u
pass=0; fail=0
ok() { echo "  ✓ $1"; pass=$((pass+ 1)); }
bad() { echo "  ✗ $1"; fail=$((fail+ 1)); }
set -a; . /root/vpn/params.env; set +a

echo "== 1. 服务状态 =="
systemctl is-active --quiet xray                && ok "xray 运行中"           || bad "xray 未运行"
systemctl is-enabled --quiet xray                && ok "xray 开机自启"         || bad "xray 未设自启"
systemctl is-active --quiet hysteria-server      && ok "hysteria 运行中"         || bad "hysteria 未运行"

```

```

systemctl is-enabled --quiet hysteria-server    && ok "hysteria 开机自启"  || bad "hysteria 未设自启"

echo "== 2. 配置语法 =="
xray run -test -config /usr/local/etc/xray/config.json >/dev/null 2>&1 \
    && ok "xray 配置合法" || bad "xray 配置有语法错误"
timeout 5 hysteria server --config /etc/hysteria/config.yaml --check >/dev/null 2>&1 \
    && ok "hysteria 配置合法" || echo "   ▲  hysteria 无 --check 参数, 跳过 (看 journalctl 报错) "

echo "== 3. 端口监听 =="
# HY2_LISTEN 形如 ":443" 或 ":20000-20100"; 取范围里的第一个端口来查
HY2_FIRST_PORT="${HY2_LISTEN#}"; HY2_FIRST_PORT="${HY2_FIRST_PORT%%-*}"
ss -tlnp | grep -qE ":{VLESS_PORT}[:space:]" && ok "TCP ${VLESS_PORT} 已监听 (VLESS)" || bad
↪ "TCP ${VLESS_PORT} 未监听"
ss -ulnp | grep -qE ":{HY2_FIRST_PORT}[:space:]" && ok "UDP ${HY2_FIRST_PORT} 已监听 (hy2)" ||
↪ bad "UDP ${HY2_FIRST_PORT} 未监听"

echo "== 4. 防火墙 =="
ufw status | grep -q "${VLESS_PORT}/tcp"    && ok "ufw 放行 ${VLESS_PORT}/tcp" || bad "ufw 未放行
↪ ${VLESS_PORT}/tcp"
# 端口跳跃时整段都要放行; 单端口就是那个端口
if [ "${HY2_FIRST_PORT}" = "${HY2_LISTEN#}" ]; then
    ufw status | grep -q "${HY2_FIRST_PORT}/udp" && ok "ufw 放行 ${HY2_FIRST_PORT}/udp" || bad "ufw
↪ 未放行 ${HY2_FIRST_PORT}/udp (hy2 用) "
else
    ufw status | grep -q "${HY2_FIRST_PORT}:${HY2_LISTEN#}/udp" && ok "ufw 放行 ${HY2_LISTEN#}
↪ 整段 udp" || bad "ufw 未放行端口跳跃范围 ${HY2_LISTEN#} (hy2 用) "
fi
echo "   i   别忘了确认云服务商安全组也放行了! "

echo "== 5. 出网 =="
curl -sS --max-time 8 https://api.ipify.org && echo "   ✓ 服务器出网正常 (IP 如上) " || bad
↪ "服务器无法出网"

echo "== 6. REALITY 目标站 =="
xray tls ping "${REALITY_TARGET}" 2>&1 | grep -qiE 'TLS 1\.\.3' \
    && ok "${REALITY_SNI} 支持 TLS 1.3" || bad "${REALITY_SNI} 不支持 TLS 1.3, 换个 target"

echo "== 7. 证书 =="
echo | openssl s_client -connect "${DOMAIN}:443" -servername "${DOMAIN}" 2>/dev/null \
    | openssl x509 -noout -dates 2>/dev/null | sed 's/^/      /' \
    || echo "   ▲  取不到证书信息 (自签证书时需用 pinSHA256) "

echo "== 8. 时间同步 (REALITY 硬性要求) =="
timedatectl show -p NTPSynchronized --value | grep -q yes && ok "时间已同步" || bad "时间未同步, 跑
↪ timedatectl status 排查"

echo
echo "===== 通过 ${pass} 项, 失败 ${fail} 项 ====="
[ "$fail" -eq 0 ]
SH

# 补一个变量给脚本用
sed -i "s/^HY2_LISTEN=.*&\nHY2_PORT_HINT=\"443\"/" /root/vpn/params.env

chmod +x /root/vpn/check.sh
/root/vpn/check.sh

```

代码已另存为 [snippets/58-9.-验收清单.sh](#) (61 行, bash)

客户端侧的最终验收:

```
# 1. 出口 IP 是否变成了服务器的 IP
# 配置好代理后, 浏览器访问 https://ip.sb 或 https://api.ipify.org
# 2. 分别单独测两个节点
# - 只开 REALITY: 能通 → TCP 通道 OK
# - 只开 hy2: 能通 → UDP 通道 OK
# 3. 测速 https://www.cloudflare.com/speedtest/
# 4. 晚高峰 20:00-23:00 再测一次, 这个时段才见真章
```

代码已另存为 [snippets/59-9.-验收清单.sh](#) (7 行, bash)

1.12 10. 排错手册

按这个顺序排查, 不要一上来同时改多处: 配置语法 → 服务状态 → 端口监听 → 防火墙 → 云安全组 → DNS → 客户端参数

1.12.1 墙内环境专项 (先看这张表)

GFW 环境下最常见的几类问题, 以及它们真正的成因:

症状	真实成因	处置
hy2 时好时坏, 晚高峰必卡	墙内对 UDP/QUIC 的 QoS 与丢包	这是墙内最普遍的现象。别调配置, 直接切到 VLESS+REALITY 线路; 或去掉 bandwidth 改用 BBR (见 §5.3)
hy2 完全连不上, 但 REALITY 正常	UDP 被封/被限死	正常现象, 墙内常态。确认 REALITY 线路能上网就够了, hy2 当加速用, 挂了不影响
REALITY 也突然连不上了	VPS 的 IP 被 GFW 封了	换 IP (重装系统换 IP, 或换服务商)。换 IP 比调配置有用得多
客户端显示连上但网页打不开	DNS 没走代理 / 被污染	检查客户端是否开启「远端 DNS」; REALITY 本身不需要域名, 但仍要确保客户端的 DNS 走代理 (§2.1 讲了 socks5h 的道理)
只有某些网站打不开, 其它正常	本机 DNS 被污染, 命中了特定域名	改的是本机的 DNS, 不是服务器上的 (服务器在墙外, 它查到的本来就是对的)。在客户端里把 DNS 设成「远端/代理」或直接指定 1.1.1.1 / 8.8.8.8, 让查询交给 VPS 去做
SSH 变慢或超时	跨境链路拥塞 / 该 IP 的 22 端口被 QoS	换高位端口 (§2.6); 或换 VPS
SSH 彻底不通	IP 被封	只能换 IP。记住 SSH 是你的逃生通道, 别在部署期间把它改坏 (§3.2)
节点白天好、晚上就断	晚高峰跨境拥塞 / UDP QoS 加剧	用 REALITY (TCP); hy2 可考虑端口跳跃 (见 §5.3 端口跳跃)
同一节点换个网络就好	你当前网络的 ISP 有针对性 QoS	换网络 (手机流量 vs 宽带常表现不同); 或换 IP

判断「配置错了」还是「被墙了」的快速方法: 在服务器上 `curl -s https://api.ipify.org` —— 能通说明服务器本身没问题, 那客户端连不上就是墙内链路或 IP 的问题, 跟你的配置无关。

这条命令能帮你避免在配置上无谓地浪费时间。

1.12.2 通用

症状	最可能的原因	处置
systemctl status 显示反复重启	配置语法错 / 端口被占	<code>journalctl -u xray -n 50 --no-pager</code> 看真实报错; <code>ss -tlnp grep 443</code> 查占用
服务正常但客户端超时一会儿通一会儿断	防火墙 / 安全组 目标站不稳定 / IP 被 QoS	<code>ufw status verbose</code> + 云控制台两边都查 换 target; 换 IP
重启 VPS 后失效	没设自启	<code>systemctl enable --now xray hysteria-server</code>
全都配好了但就是慢	线路问题, 不是配置问题	换服务商/换地区, 别折腾配置

1.12.3 VLESS + REALITY 专项

症状	原因	处置
握手失败 / reality verification failed	客户端和服务端参数对不上	逐字比对 UUID、pbk(password)、sid、SNI、fp 五项， 尤其确认 pbk 填的是 Password (PublicKey) 那一行，不是 PrivateKey
sid 报错	shortId 长度非法	必须是偶数长度、≤16、只含 0-9a-f。a1b2c3 (奇数) 非法
之前能用，升级 Xray 后突然全挂	新版 REALITY 有客户端版本下限	老客户端被静默重定向到 target 站， 不报错、只是连不上 。 (1) 升级客户端；(2) 或在 realitySettings 里加 "min-ClientVer": "25.9.11" 放宽限制
探测端口看到的是真网站	这是 REALITY 的 预期行为 ，不是 bug	不用管
你的服务器被扫描后流量被白嫖	target 是 Cloudflare 这类大 CDN	换非 CDN 大站，或配 limitFallbackUpload/Download 限速
启动时出现 apple/非 443 端口警告	Xray v26.3.27 新增的预警	换 target、换回 443。这类配置极易导致 IP 被封
服务端日志刷 geoip 相关错误	装的时候跳过了 geo-data	bash -c "\$(curl -L https://github.com/XTLS/Xray-install/raw/main/install-release.sh)" @ install-geodata

1.12.4 Hysteria 2 专项

症状	原因	处置
tls: handshake failure	客户端 SNI 与证书不匹配	SNI 必须等于 acme.domains 里的域名
certificate verification failed	域名没解析 / 证书过期	dig +short 域名确认指向本机；看 journalctl 里 ACME 报错
authentication failed	auth 密码填错	重新 cat /etc/hysteria/config.yaml 核对
连接建立后立刻断，日志无明显报错	obfs 密码填错 (hy2 的典型症状：静默拒绝)	核对 obfs.salamander.password
能 ping 通域名但连不上	Cloudflare 开了橙云	改成 DNS only (灰云)。Cloudflare 常规代理不转发 hy2 的 UDP 流量
证书申请失败	80 端口不通 / LE 频率限制	放行 80/tcp (ufw + 安全组)；或改 DNS-01；或等限流恢复；或用自签
开了端口跳跃后时断时续	防火墙没放行整个范围	ufw allow 20000:20100/udp 且云安全组也要放行；仍不行就 退回单端口 443
开了端口跳跃后日志报权限错	需要 root 或 CAP_NET_ADMIN	官方 unit 已带 AmbientCapabilities=CAP_NET_ADMIN，确认没被自己改掉；或直接退回单端口
hy2 握手失败 + 服务端用的是 Ed25519 证书	sing-box 1.14+ 客户端模拟 Chrome QUIC 指纹，不宣告 Ed25519	换成 ECDSA (P-256) 或 RSA 证书
速度忽快忽慢，晚高峰必卡	运营商 UDP QoS	这是 hy2 的宿命。 开 BBR (去掉 bandwidth 配置) ，并保留 VLESS+REALITY 作备份

1.12.5 sing-box 专项

症状	处置
配置报错但看不出哪错	sing-box check -c config.json，它会指出具体字段；sing-box 对未知字段直接报错，能帮你避开版本差异
用了 acme 报废弃警告	1.14+ 改用 certificate_provider；内联写法 1.16 移除
端口跳跃不生效	确认用的是 server_ports (1.11+)，不是 server_port；服务端也要真的开了范围

1.13 11. 日常运维

1.13.1 升级

```
# Xray (官方脚本, 升级时会保留配置)
bash -c "$(curl -L https://github.com/XTLS/Xray-install/raw/main/install-release.sh)" @ install
xray version && systemctl restart xray

# Hysteria 2 (重跑安装脚本即可, 会自动重启运行中的服务)
bash <$(curl -fsSL https://get.hy2.sh/)
systemctl status hysteria-server

# sing-box
curl -fsSL https://sing-box.app/install.sh | sh
systemctl restart sing-box
```

代码已另存为 [snippets/60-升级.sh](#) (11 行, bash)

▲ Xray 大版本升级前, 先用 minClientVer 想清楚客户端兼容问题。参见上面排错表里那条「升级后突然全挂」。

1.13.2 备份

```
tar czf /root/vpn-backup-$(date +%F).tar.gz \
/usr/local/etc/xray/config.json \
/etc/hysteria/config.yaml \
/root/vpn/params.env \
/var/lib/hysteria/          # hy2 的 ACME 证书
```

代码已另存为 [snippets/61-备份.sh](#) (5 行, bash)

把备份放在你本地, 不要只留在服务器上。服务器被封/被回收时, 私钥和密码一起丢就完了。

1.13.3 加用户

```
# Xray: 多一个 UUID
xray uuid
# 把新 UUID 追加到 config.json 的 settings.clients 数组里, 然后 restart
# 每个客户端给不同的 shortId, 方便以后精确踢掉某一个
```

代码已另存为 [snippets/62-加用户.sh](#) (4 行, bash)

1.13.4 加端口 (抗封备用)

REALITY 本身不支持一个端口监听多端口的写法之外的花活, 但可以:

- 改 VLESS_PORT 到 8443 后重启 (注意 Xray 会警告非 443 端口风险变高);
- 客户端配多个节点做 url-test 自动切换;
- 换 IP (IP 被封是常态, 换 IP 比换协议有效得多)。

1.13.5 加流量限制 (多人共用时)

hy2 用 bandwidth; Xray 侧可以加路由规则或用面板 (3X-UI / Remnawave / Marzban 等) 做流量统计和限速。

1.14 12. 别做的事 (会显著提高被封概率或造成安全事故)

协议配置层面

1. **x 用 www.apple.com 作 REALITY target** —— Xray v26.3.27 已明确警告, 实测极易导致 IP 被封。老教程里这个值满天飞。
2. **x REALITY 用非 443 端口** —— 同样被官方点名为高风险行为。
3. **x Vision 和普通 TLS 代理混用** (比如同一个端口/同一套配置里再开个 VLESS+TLS) —— 抗封优势直接归零。
4. **x 不开客户端 uTLS 指纹** —— 官方列出的「正确配置」第 (4) 条。
5. **x 不禁回国流量** —— 服务器主动连国内 IP 是显著特征。
6. **x 只有 hy2 没有 TCP 备份** —— 运营商 UDP QoS 会让你在晚高峰彻底断网。
7. **x REALITY 回落/混用其它代理协议** —— 回落必须是普通网页。

安全层面

8. **x 私钥泄露** —— privateKey 只在服务端。看到别人要你的私钥, 一律不给。REALITY 的设计就是客户端拿的是 Password (PublicKey), 永远不要把服务端私钥填进任何客户端。

9. **x 3X-UI 等管理面板裸露在公网** —— 安装完立刻改面板端口 + 改默认用户名密码 + 最好限制来源 IP。面板被扫 = 你的所有节点被接管。
10. **x 弱密码** —— hy2 的 auth / obfs 密码都用 openssl rand -hex 24 生成，别用生日。
11. **x 在 params.env / creds.txt 写完不 chmod 600** —— 里面全是密钥。
12. **x 直接用 curl | bash 跑来路不明的脚本** —— 本文所有脚本 URL 都指向官方仓库。自己核对一遍再执行。
13. **x 用第三方 GitHub 镜像下载代理软件** —— 镜像方能看到你的 UUID、密钥、订阅链接。

认知层面

14. **x 以为配对了就 100% 不被封** —— 官方原话：无法保证 IP 干净、无法避免被邻居波及、无法避免整个 IP 段被重点照顾。被封了就换端口 → 换 IP → 换服务商依次试，不要怀疑人生。
15. **x 同时排查多处配置** —— 一次只改一个地方，否则你不知道是哪个改动生效的。

1.15 13. 参考资料（官方一手来源）

内容	地址
Xray-core 仓库 / Releases	https://github.com/XTLS/Xray-core
Xray 官方安装脚本说明	https://github.com/XTLS/Xray-install
REALITY 官方配置示例（服务端 + 客户端 json5）	https://github.com/XTLS/REALITY
Xray 配置文档（REALITY 字段详解）	https://xtls.github.io/config/transport/reality.html
Xray 官方示例（VLESS-TCP-XTLS-Vision）	https://github.com/XTLS/Xray-examples
Hysteria 2 服务端部署	https://v2.hysteria.network/docs/getting-started/Server/
Hysteria 2 完整服务端配置	https://v2.hysteria.network/docs/advanced/Full-Server-Config/
Hysteria 2 端口跳跃	https://v2.hysteria.network/docs/advanced/Port-Hopping/
Hysteria 2 官方安装脚本源码	https://get.hy2.sh/
sing-box 安装	https://sing-box.sagernet.org/installation/
sing-box VLESS 出站	https://sing-box.sagernet.org/configuration/outbound/vless/
sing-box Hysteria2 出站	https://sing-box.sagernet.org/configuration/outbound/hysteria2/
sing-box TLS / REALITY / uTLS 字段	https://sing-box.sagernet.org/configuration/shared/tls/
Mihomo（Clash.Meta 内核）	https://github.com/MetaCubeX/mihomo

网上中文教程大量停留在 2023-2024 年的字段命名（dest、publicKey、network: tcp、up/down）。
照抄老教程是本指南第一节列的那些坑的主要来源。有争议时以官方文档为准。

1.16 14. 官方资料离线存档

上表 14 个链接的官方原文已经全部下载到本仓库的 **references/ 目录**，断网也能查、也能直接复制配置。

目录	内容	对应章节
references/hysteria/	Hysteria 2 服务端部署、完整配置字段、端口跳跃（3 篇）	\$5、\$7.2
references/sing-box/	VLESS / Hysteria2 / TLS 出站字段， 含官方中文版 ；安装方式	\$8
references/xray/	REALITY 传输层字段详解、REALITY 原理、安装脚本说明	\$3、\$6
references/examples/	Xray 官方示例 config_server/client.jsonc（REALITY、Vision、防 RST、Hysteria2 客户端）	\$6、\$9.2
references/mihomo/、references/v2rayN/	客户端说明	\$9.1、\$9.3

目录	内容	对应章节
references/scripts/	本指南里被直接执行的 4 个官方脚本原文 + GPG 公钥	§3.1、§5.1、 §8.0

看完整索引（含每份文件的来源 *URL*、上游 *commit*）

`less references/INDEX.md`

校验本地文件与抓取时是否一致（*SHA256* + 字节数）

`python3 fetch_references.py --verify`

上游文档更新后重新抓取

`python3 fetch_references.py`

代码已另存为 `snippets/63-14.-官方资料离线存档.sh`（8 行，bash）

几点说明：

- **这是快照，不是镜像。**每份文件都记了抓取日期和上游 commit（见 `references/MANIFEST.json`），要最新内容以官方站点为准。
- Hysteria 那 3 篇是从官方渲染页转的 Markdown（官方未公开文档源码仓库），代码块已还原成围栏格式，官方被 Cloudflare 隐藏的示例邮箱也已解码，可直接复制。
- **抄示例配置前必须替换密钥。**官方示例里的 `id / password / privateKey / shortId / password` 全是公开示例值，原样照抄等于没加密。
- 版权归原作者所有（Xray/REALITY 为 MPL-2.0，Hysteria/sing-box 为 GPL-3.0），本目录仅作学习存档。